

BCSE307P COMPILER DESIGN LAB PROJECT

(Slot: G2+TG2)

AUTOMATED LLM/AGENT-GUIDED COMPILER CODE OPTIMIZATION

Submitted by:

Registration Number	Name
24BCE2972	Adarsh Sharma Badhai (Team Leader)
24BAI0345	Nirupam Raut

Under the guidance of:

Prof. Mukhtar Ahmad Sofi

School of Computer Science and Engineering (SCOPE)

October 2026

ABSTRACT

Standard compilers apply a fixed chain of optimization passes, rarely confirming if a specific transformation actually improved the code's efficiency. We built a fully automated, multi-agent system to optimize Three-Address Code (TAC) dynamically. Instead of a static pipeline, an orchestrator loop analyzes the code, ranks available opportunities, and proposes transformations using either a rule-based engine or a Google Gemini LLM fallback. Two more LLMs are supported behind the same interface: Groq in the cloud, and a local model served by Ollama that works with no internet connection. To guarantee correctness, every proposed change must pass a strict dual-verification gate: differential testing across 25+ input sets and symbolic polynomial equivalence for straight-line logic. If a candidate passes, a static cost model evaluates the precise performance benefit before accepting it. Tested on a dataset of 500 programs spanning seven structural categories, the rule-based agent alone achieved an 82.5% cost reduction. Integrating Gemini pushed this to 91.5%, largely by successfully rewriting complex loops and conditionals that standard rules missed. Groq reached 90.9% and the local model 83.3%. The dual-verification system successfully blocked all 26 of Gemini's incorrect suggestions, proving that language models can be safely integrated into compiler optimization when constrained by formal equivalence checks, resulting in a zero false-positive rate. It also blocked all 6 incorrect suggestions from Groq and all 113 from the local model, so every run finished with zero false positives.

TABLE OF CONTENTS

1 INTRODUCTION **5**

1.1 Background 5

1.2 Motivation 6

1.3 Scope of the Project 7

2 PROJECT DESCRIPTION AND GOALS **9**

2.1 Literature Review 9

2.1.1 Classical Compiler Optimization 9

2.1.2 Superoptimization and Stochastic Search 9

2.1.3 Translation Validation 10

2.1.4 Large Language Models for Code Transformation 10

2.1.5 Multi-Agent Architectures 10

2.2 Gaps Identified 10

2.3 Objectives 11

2.4 Problem Statement 12

2.5 Project Plan 13

3 TECHNICAL SPECIFICATION **14**

3.1 Requirements 14

3.1.1 Functional Requirements 14

3.1.2 Non-Functional Requirements 15

3.2 System Specification 16

3.2.1 Hardware Specification 16

3.2.2 Software Specification 16

4 DESIGN APPROACH AND DETAILS **17**

4.1 Datasets 17

4.2 System Architecture 18

4.2.1 Routing Between the Rules and the LLM 19

4.2.2 LLM Backends 19

4.3 Algorithms 20

4.3.1 Liveness Analysis (Backward May-Analysis) 20

4.3.2 Available-Copy Analysis (Forward Must-Analysis) 20

4.3.3 Symbolic Polynomial Normalization 20
4.3.4 Differential Testing 21
4.4 Design 22
4.4.1 Data Flow Diagram 22
4.4.2 Class Diagram 22

5 IMPLEMENTATION **23**
5.1 Module Description 23
5.1.1 Module 1: Front End (`src/frontend/`) 23
5.1.2 Module 2: TAC Representation and Interpreter (`src/tac/`) . . 23
5.1.3 Module 3: Agent Components (`src/agents/`) 24
5.1.4 Module 4: Optimization Specialist and Routing 25
5.1.5 Module 5: LLM Optimizer (`src/agents/optimizer_llm.py`) 26
5.1.6 Module 6: Visualization and Demo 26
5.2 Testing 26

6 RESULTS AND DISCUSSION **29**
6.1 Detailed Results 29
6.1.1 Comparing the Three LLMs 30
6.2 Output Screenshots 31
6.2.1 Sample Run 31
6.2.2 Sankey Diagram: Proposal Flow 32
6.2.3 Grouped Bar Chart: Outcomes by Optimization Type 33
6.2.4 Line Chart: Cost Trajectory over Iterations 33
6.2.5 Pie Chart: Accepted Optimization Types 34
6.2.6 Scatter Plot: Steps vs. Cost Improvement 34
6.2.7 Heatmap: Category vs. Optimization Type 35
6.2.8 Comparison Charts: Rule-Only vs. Rule + LLM 35
6.2.9 Analysis Charts for the Groq Run 36
6.2.10 Analysis Charts for the Ollama Run 39
6.3 Performance Analysis and Evaluation 42
6.3.1 Limitations 44

REFERENCES **45**

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Compiler optimization sits at the boundary between language semantics and machine performance. Since the 1970s, production compilers have shipped with a fixed menu of transformation passes (constant folding, dead-code elimination, loop-invariant code motion, register allocation, and so on) executed in a hardwired order chosen by the compiler engineers. GCC, for instance, runs over two hundred passes in its `-O2` pipeline (Stallman et al., 2023). LLVM follows a similar philosophy, chaining dozens of analysis and transform passes into a predetermined sequence (Lattner & Adve, 2004).

The trouble with a fixed pipeline is that it cannot adapt. A program dominated by redundant subexpressions might benefit from aggressive common-subexpression elimination early on, while a program littered with dead stores would be better served by liveness-based cleanup first. The compiler has no way to reason about which pass to run next, or to judge whether a transformation it just applied actually made things better.

Two lines of research have tried to address this rigidity. The first is superoptimization, pioneered by Massalin in 1987 and later scaled up by the STOKE project (Massalin, 1987; Phothilimthana et al., 2016). Superoptimizers search exhaustively (or stochastically) for the shortest instruction sequence that produces the same output, and they verify candidates through testing or SMT solving. They work well on tiny basic blocks but run into combinatorial walls on anything larger.

The second line comes from the recent wave of large language models applied to code. Codex and Code Llama showed that transformer-based models can generate syntactically correct programs from natural-language descriptions (Chen et al., 2021; Rozière et al., 2023). Meta's LLM Compiler project trained models specifically on LLVM IR and used them to predict which pass ordering would yield the best binary (Cummins et al., 2024). CompilerGym framed pass selection as a reinforcement-learning problem (Cummins et al., 2022). But all of these treat the compiler as a black box whose knobs

are being tuned; none of them propose new code-level transformations and verify them against the original semantics.

Translation validation, introduced by Pnueli, Siegel, and Singerman and later refined by Necula, takes the opposite approach (Necula, 2000; Pnueli et al., 1998). Rather than trusting the compiler, it checks after the fact that the output program is semantically equivalent to the input. The idea is sound, but it has mostly been applied as a post-hoc auditor, not as an online decision-maker that guides the optimization process.

Our project brings these threads together. We build a multi-agent system whose orchestrator loop reasons about what to optimize, delegates the actual transformation to either a rule-based engine or an LLM, verifies every candidate through differential testing and symbolic equivalence, and only commits changes that are verified correct and measurably cheaper. The agent runs autonomously from start to finish; no human touches the intermediate code.

1.2 MOTIVATION

We started this project out of a straightforward frustration: compilers are not transparent about why they apply a particular optimization, and they never explain whether a transformation they skipped might have been worthwhile. For a Compiler Design course, understanding optimization is at least as important as implementing parsing and code generation, yet most course projects stop at the TAC generation stage and never close the loop back to correctness.

Two developments made the idea feasible. First, free LLMs are now easy to reach. Our first prototype used a paid API, which made a 500-program run impractical, so we moved to options that cost nothing: Google Gemini and Groq on their free tiers, and a model that runs on our own laptop through Ollama. The local model also removes the dependence on an internet connection and on a daily quota. Second, differential testing is cheap: running a TAC interpreter on a few hundred input vectors takes a fraction of a second per program, so a brute-force equivalence check fits comfortably inside an iterative loop.

The gap we wanted to fill was the absence of any system that pairs an LLM proposer with a verification gate. LLMs hallucinate; that is well documented. But if every sug-

gestion is tested before it is accepted, the hallucination rate becomes a measurable rejection rate rather than a silent bug rate. We thought that would be an interesting thing to quantify, and the results bore it out: 7.65% of Gemini's proposals, 1.9% of Groq's and 57% of the local model's turned out to be wrong, and the verifier caught all of them.

1.3 SCOPE OF THE PROJECT

The project covers the following:

- A complete front-end pipeline (lexer, recursive-descent parser, AST, TAC generator) for a small C-like language with integers, arithmetic, comparisons, logical operators, `if`/`else`, `while`, `for`, and `print`, and a second front end that accepts a subset of real C for the live demo.
- A TAC interpreter that can execute programs with user-supplied input variables, including division-by-zero trapping and loop-timeout detection.
- Dataflow analyses on the instruction-level control-flow graph: liveness, available copies, and definite assignment.
- Six rule-based optimization passes: constant folding, algebraic simplification, dead-code elimination, common-subexpression elimination, strength reduction, and copy propagation.
- An Optimization Specialist that routes every step to the rule-based optimizer or to an LLM, in one of three modes (`rule`, `hybrid`, `llm`).
- Three LLM proposers behind one interface: Google Gemini, Groq, and a local model served by Ollama for offline use, with structured prompting, response parsing, and automatic retry or disable on API errors.
- A two-stage verifier combining differential testing (random, edge, boundary, small-scope and pairwise input sets) and symbolic polynomial equivalence checking.
- A weighted static cost model used as the accept/reject gate.
- An orchestrator that ties these components into an autonomous loop running up to 50 iterations per program.

- A 500-program dataset covering seven structural categories (arithmetic, nested, repeated-expression, dead-code, loops, conditionals, mixed).
- Seven visualization charts and a per-program decision log recording every step the agent took, including every routing decision.

What the project does not cover:

- Backend code generation (register allocation, instruction selection, machine code emission).
- Interprocedural or whole-program optimization (function inlining, call-graph analysis).
- Floating-point arithmetic, arrays or string types.
- SMT-based formal verification via Z3 (we rely on differential testing and symbolic equivalence instead).
- Measured execution time: the cost model is static.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

2.1.1 Classical Compiler Optimization

The Dragon Book (Aho et al., 2006) remains the standard reference for dataflow-driven optimization. It describes the lattice-theoretic framework underpinning constant propagation, liveness analysis, and available-expression analysis, and it lays out the algorithms for dead-code elimination, common-subexpression elimination, and loop-invariant code motion. Our implementation follows the Dragon Book's formulation of liveness and available-copy analysis closely, adapting the iterative fixed-point algorithms to operate at the instruction level rather than the basic-block level.

Muchnick's textbook (Muchnick, 1997) goes deeper into SSA form, partial redundancy elimination, and global value numbering. We did not adopt SSA (our TAC stays in a simpler named-variable form), but Muchnick's treatment of copy propagation and algebraic simplification informed several of our rule-based transformations.

2.1.2 Superoptimization and Stochastic Search

Massalin's 1987 superoptimizer (Massalin, 1987) was the first system to enumerate all possible instruction sequences of increasing length and test each one against the original program. The approach is exact but impractical beyond five or six instructions. STOKE (Phothilimthana et al., 2016) replaced exhaustive search with Markov-chain Monte Carlo sampling, trading completeness for scalability. STOKE verifies candidates using a combination of test-case evaluation and, optionally, an SMT solver. Our differential-testing verifier is conceptually similar to STOKE's test-case phase, though we add symbolic polynomial normalization as a second check.

2.1.3 Translation Validation

Pnueli et al. introduced translation validation as a way to verify each run of a compiler rather than verifying the compiler itself (Pnueli et al., 1998). Necula extended the idea to handle optimizations like loop unrolling and strength reduction by constructing a simulation relation between the source and target programs (Necula, 2000). Our symbolic verifier is a lightweight variant of this idea: it traces both programs symbolically and compares canonical polynomial representations of the printed output expressions.

2.1.4 Large Language Models for Code Transformation

Codex (Chen et al., 2021) demonstrated that language models trained on code can solve programming problems with surprisingly high accuracy. Code Llama (Rozière et al., 2023) extended this to open-weight models fine-tuned on code-heavy corpora. More recently, Meta's LLM Compiler (Cummins et al., 2024) fine-tuned Code Llama on 546 billion tokens of LLVM IR and assembly, training it to predict optimized IR and optimal pass orderings. CompilerGym (Cummins et al., 2022) framed compiler optimization as a reinforcement-learning environment. Our work differs from these in a fundamental way: we use the LLM not to pick a pass ordering but to directly propose a rewritten TAC program, and we verify the proposal before accepting it. Open-weight code models can also be run locally (Ollama, 2026), which lets the same proposer work with no internet connection.

2.1.5 Multi-Agent Architectures

AutoGen (Wu et al., 2023) showed that multi-agent conversations, where different agents take on different roles, can solve complex tasks more reliably than a single monolithic prompt. CrewAI and similar frameworks have since popularized the pattern for code generation and research workflows. Our orchestrator–specialist architecture borrows from this idea: the orchestrator does not itself know how to fold constants or eliminate dead code; it delegates to specialist agents and arbitrates their output through the verifier and cost evaluator.

2.2 GAPS IDENTIFIED

After surveying the literature, we identified the following gaps:

1. Classical compilers do not reason about which pass to apply next or whether a transformation was actually beneficial. The pass ordering is hardwired.
2. Superoptimizers handle small code fragments well but cannot scale to programs with loops and branches.
3. LLM-based code tools produce plausible output but provide no correctness guarantee. An LLM can silently change program semantics, and the user has no way to know without manual inspection.
4. Existing LLM-compiler work focuses on pass ordering or flag selection, not on proposing new code-level transformations.
5. No prior system combines an LLM-based transformation proposer with a formal or semi-formal verification gate that blocks incorrect suggestions automatically.
6. LLM-based tools usually depend on one cloud provider: without a connection, a key or remaining quota they stop working.

2.3 OBJECTIVES

1. Build a multi-agent architecture with five clearly separated components: an orchestrator, a code analysis specialist, an optimization specialist (rule-based and LLM-based), a verification module, and a cost evaluator.
2. Implement the orchestrator loop: analyze, propose, verify, evaluate, accept or reject, and repeat until no opportunity remains.
3. Detect six types of optimization opportunities (constant folding, algebraic simplification, dead-code elimination, common-subexpression elimination, strength reduction, copy propagation) and apply the highest-priority one at each step.
4. Implement dual verification: differential testing on many input sets and symbolic polynomial equivalence for straight-line code.
5. Accept a transformation only when both verification and cost evaluation confirm it is correct and beneficial.

6. Integrate free LLMs (Google Gemini and Groq) as proposers, with the same verification gates as rule-based proposals, and let one component decide for every step whether the rules or the LLM should propose.
7. Keep the system usable with no internet connection by supporting a local model.
8. Run the agent on 500 programs across seven categories and report verification pass rate, acceptance rate, cost reduction, convergence steps, LLM validity rate, and false-positive rate.
9. Produce seven visualization charts summarizing the agent's behavior across the dataset.

2.4 PROBLEM STATEMENT

Given a program written in a C-like source language, automatically lower it to Three-Address Code and then iteratively optimize the TAC through an autonomous agent loop that (a) identifies optimization opportunities using dataflow analysis, (b) proposes a single transformation per iteration, either rule-based or LLM-generated, as decided by a router, (c) verifies the candidate against the current program through differential testing and symbolic equivalence, (d) accepts the change only if the weighted cost strictly decreases, and (e) repeats until convergence. The system must achieve an average cost reduction above 30% across a 500-program dataset, with a false-positive rate of exactly zero, and it must keep working when no LLM can be reached.

2.5 PROJECT PLAN

Table 2.1: Project timeline and milestones

Phase	Task	Duration
1	Literature survey; define source language grammar; design TAC instruction set	Weeks 1–2
2	Implement lexer, parser, AST, TAC generator, and TAC interpreter	Weeks 3–4
3	Implement dataflow analyses (CFG, liveness, available copies, definite assignment)	Week 5
4	Build the five agent components (analyzer, rule optimizer, verifier, cost evaluator, decision logger)	Weeks 5–7
5	Wire agents into the orchestrator loop; run on small test programs; debug	Week 7
6	Generate the 500-program dataset; run the rule-based agent on all programs; collect metrics	Week 8
7	Integrate Google Gemini LLM proposer; run the combined agent; compare with rule-only baseline	Week 9
8	Build visualization charts; write decision-log exporter	Weeks 9–10
9	Add the Optimization Specialist router; add the Groq and Ollama backends and the offline mode	Week 11
10	Test the verifier itself (fuzzing, mutation testing); add small-scope and pairwise input sets	Week 12
11	Full runs with all three LLMs; C front end and live demo; report	Weeks 12–13

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

1. **Source-to-TAC compilation:** The system shall accept programs written in the defined C-like language and produce correct Three-Address Code through lexing, parsing, and code generation.
2. **Optimization opportunity detection:** The code analyzer shall identify constant-folding, algebraic-simplification, dead-code-elimination, common-subexpression, strength-reduction, and copy-propagation opportunities from the TAC, using dataflow analysis on the instruction-level control-flow graph.
3. **Routing and transformation proposal:** The Optimization Specialist shall route each step to the rule-based optimizer or to an LLM and record the reason. The rule-based optimizer shall apply exactly one transformation per iteration. The LLM optimizer shall, when invoked, send the current TAC to the configured model (Gemini, Groq or a local Ollama model) and parse the returned candidate.
4. **Dual verification:** Every candidate (rule-based or LLM-generated) shall pass differential testing on at least 25 input sets and, for straight-line code, symbolic polynomial equivalence checking, before being considered for acceptance.
5. **Cost-based acceptance:** A candidate shall be accepted only if its weighted static cost is strictly lower than the current program's cost.
6. **Iterative convergence:** The orchestrator shall repeat the loop until no opportunity remains or the iteration cap (50) is reached.

7. **Decision logging:** Every iteration shall be logged with the opportunity analyzed, the route chosen, the transformation proposed, the verification outcome, the cost change, and the accept/reject decision. Logs shall be saved in both human-readable text and structured JSON.

8. **Metrics and visualization:** The system shall compute verification pass rate, acceptance rate, average cost reduction, convergence steps, LLM validity rate, and false-positive count, and produce seven charts.

3.1.2 Non-Functional Requirements

1. **Correctness guarantee:** The false-positive rate must be exactly 0%. No incorrectly optimized program shall pass the verification gate.

2. **Modularity:** Each agent component shall be a separate Python class with a documented input/output interface, so that new optimizations, verification methods or LLM providers can be added without modifying other components.

3. **Performance:** A single program shall be optimized in well under a second with the rules alone and in a few seconds with a cloud LLM. The full 500-program dataset shall complete in under 3 minutes without an LLM and in under 45 minutes with a cloud LLM.

4. **Graceful degradation:** If the LLM is unavailable (missing key, quota exhaustion, network failure, local server stopped), the system shall fall back to rule-based optimization and log the reason.

5. **Offline operation:** The whole agent, including an LLM proposer, shall be able to run with no internet connection.

6. **Reproducibility:** The dataset generator shall produce identical programs when run with the same parameters. Metrics and logs shall be saved in machine-readable formats.

3.2 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

Table 3.1: Hardware configuration used for development and testing

Component	**Specification**
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent (the runs were made on a 13th-generation Intel Core i5 laptop)
RAM	8 GB minimum; 16 GB for the local model
Storage	500 MB free disk space; a further 4.7 GB for the local model
Network	Internet connection for the Gemini and Groq calls only; none in offline mode

3.2.2 Software Specification

Table 3.2: Software stack

Software	**Version**	**Purpose**
Python	3.10+	Core language (uses `match` statements)
google-genai	$\geq$1.0	Gemini API client
Groq API	HTTP	Called with the Python standard library; no package needed
Ollama	0.40	Local model server, with the model `qwen2.5-coder:7b`
matplotlib	$\geq$3.8	Bar, line, scatter, heatmap charts
plotly	$\geq$5.18	Interactive Sankey diagram
seaborn	$\geq$0.13	Heatmap styling
numpy	$\geq$1.26	Trend-line computation
kaleido	$\geq$0.2.1	Plotly PNG export
pytest	$\geq$8.0	Unit and integration testing

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 DATASETS

We wrote a Python generator (`dataset/generator.py`) that creates 500 source programs spread across seven categories. Each category targets a different mix of optimization opportunities, so the agent faces a range of challenges rather than the same pattern repeated.

Table 4.1: Dataset composition

Category	**Count**	**What it tests**
Arithmetic-only	60	Straight-line constant expressions (`int k = 1 + 14`)
Nested expressions	60	Multi-level parenthesized sub-expressions requiring many temporaries
Repeated expressions	60	Identical sub-expressions computed more than once (CSE targets)
Dead code	60	Variables assigned but never read or printed
Loops	80	`for`/`while` loops with loop-invariant computations and dead stores inside the body
Conditionals	80	`if`/`else` branches with algebraic identities (`x * 1`, `x + 0`)
Mixed	100	Programs combining two or more of the above in a single source file

Programs that use input variables (read-before-write) list them in an `// inputs:` header comment. The verifier feeds those variables many different values during differential testing. Programs without input variables have exactly one possible execution, so a single test run constitutes a proof of equivalence.

4.2 SYSTEM ARCHITECTURE

Figure 4.1 shows the architecture. Source code enters at the top left, passes through the front end (lexer, parser, AST, TAC generator), and arrives at the agent loop. The Orchestrator controls the cycle: it asks the Code Analysis Specialist to scan the TAC, hands the step to the Optimization Specialist, which routes it to the rules or to an LLM, has the candidate verified, has its cost evaluated, and decides whether to accept. The loop repeats until nothing is left to try. The run then ends with a final check of the optimized program against the original, and its outputs are the optimized TAC, the per-program decision log, the metrics files and the charts.

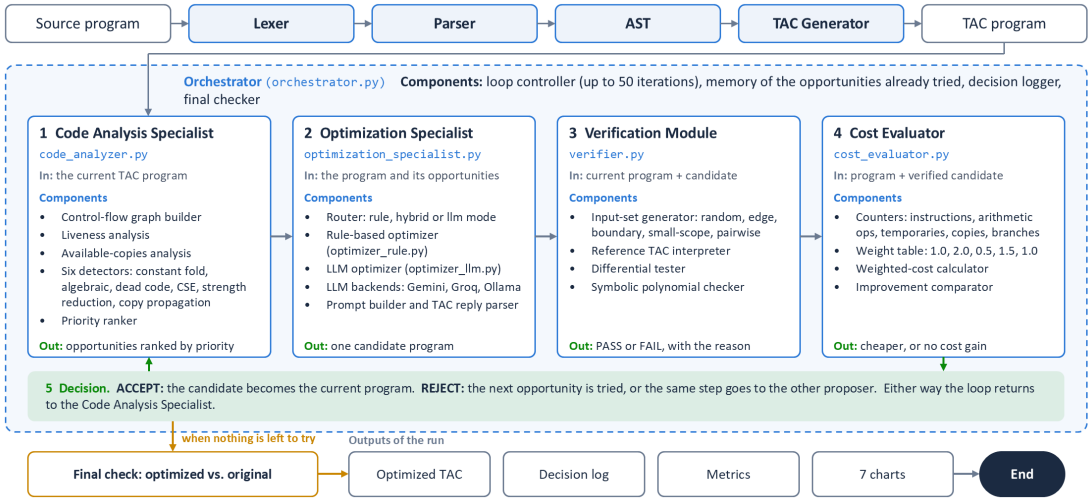


Figure 4.1: System architecture: the front end, the agent loop with the components of each agent, and the end of a run

- **Orchestrator.** Runs the loop for up to 50 iterations per program, remembers which opportunities were already turned down, logs every decision, and re-verifies the final program against the untouched original.
- **Code Analysis Specialist.** Builds the control-flow graph, runs the liveness and available-copies analyses, and returns the opportunities it finds, ranked by priority (constant folding 100, algebraic simplification 90, dead code 80, CSE 70, strength reduction 60, copy propagation 50).
- **Optimization Specialist.** Decides who proposes the next change and returns one candidate program. Option A is the rule-based optimizer, which applies the highest-priority opportunity to a copy of the program. Option B is an LLM, which receives the TAC and is asked for one semantics-

preserving optimization.

- **Verification Module.** Runs the current program and the candidate on the same inputs and compares printed output and failure mode; for straight-line code it also checks symbolic equivalence.
- **Cost Evaluator.** Computes the weighted cost of both programs and accepts the change only if the cost strictly drops.

4.2.1 Routing Between the Rules and the LLM

The Optimization Specialist has three routing modes, listed in Table 4.2. Whatever the route, the proposal goes through the same verification and cost gates. The LLM is never asked about a program that is already nothing but constant `print` instructions, because that form is provably minimal. Every routing decision is written to the decision log together with its reason.

Table 4.2: Routing modes of the Optimization Specialist

Mode	First choice	Falls back to the other proposer when
`rule`	Rules	Never: the LLM is not used
`hybrid`	Rules	The rule proposal for the step was turned down, or the rules have nothing left to try
`llm`	LLM	The LLM is not running, its call budget is used up, or its proposal was turned down

4.2.2 LLM Backends

All three backends share one prompt and one reply parser, so their results are directly comparable (Table 4.3). The provider is chosen automatically: Gemini, then Groq, then Ollama, whichever is configured and reachable. An offline switch forces the local model.

Table 4.3: The three LLM backends

Backend	Model	Runs on	Notes
Gemini	`gemini-3.1-flash-lite`	Google cloud	Free API key; called through the SDK
Groq	`gpt-oss-120b`	Groq cloud	Free API key; moves to the next model when a daily quota is spent
Ollama	`qwen2.5-coder:7b`	The laptop	No key and no internet; falls back to the CPU if the GPU runner fails

4.3 ALGORITHMS

The project relies on several well-known compiler algorithms, each adapted to work on individual TAC instructions rather than basic blocks.

4.3.1 Liveness Analysis (Backward May-Analysis)

Starting from the last instruction, we propagate live variable sets backward through the CFG. At each instruction, a variable is added to the live set if it is read, and removed if it is defined. The analysis iterates until no set changes. Dead-code elimination uses the resulting live-out sets: if an instruction defines a variable that is not live at its exit and cannot trap (no division by a variable), the instruction is dead.

4.3.2 Available-Copy Analysis (Forward Must-Analysis)

We track pairs (x, y) meaning "$x = y$ is still valid." At CFG joins we intersect, so a copy is available only if it reaches along every path. Whenever x or y is redefined, all pairs mentioning it are killed. Copy propagation uses these sets to replace uses of x with y wherever the copy is available.

4.3.3 Symbolic Polynomial Normalization

For straight-line code (no labels, no jumps), we trace each variable as a multivariate polynomial in the input variables. Addition, subtraction, and multiplication map directly to polynomial operations, so $x + x$ and $2 \times x$ normalize to the same polynomial; division and modulo produce opaque terms that are compared structurally. Two programs are symbolically equivalent if their printed expressions normalize to the same

polynomial, and the set of divisors that could trap before each print is identical.

4.3.4 Differential Testing

We run the current and the candidate TAC programs through the interpreter on the same input sets. Both programs must print the same output sequence and either both succeed or both fail with the same exception type (such as a division by zero or a timeout). The input sets are built in five ways, listed in Table 4.4.

Table 4.4: Input sets used by differential testing

Input sets	**What is tried**	**What it catches**
Random	20 sets with values from -100 to 100	General differences
Edge cases	All inputs equal to 0, 1, -1, 99, -99	Zero and sign handling
Boundary	$c-1$, c, $c+1$ for every literal c	Off-by-one comparisons
Small scope	Every combination of a few small values (one input: every value from -128 to 128)	`&&` against `||`, a branch on a flag
Pairwise	Every two inputs with every two boundary values	Two inputs that must line up

The last two kinds were added after mutation testing (Section 5.2) showed that random values almost never make one input zero while another is not, which is exactly what tells `&&` from `||`.

4.4 DESIGN

4.4.1 Data Flow Diagram

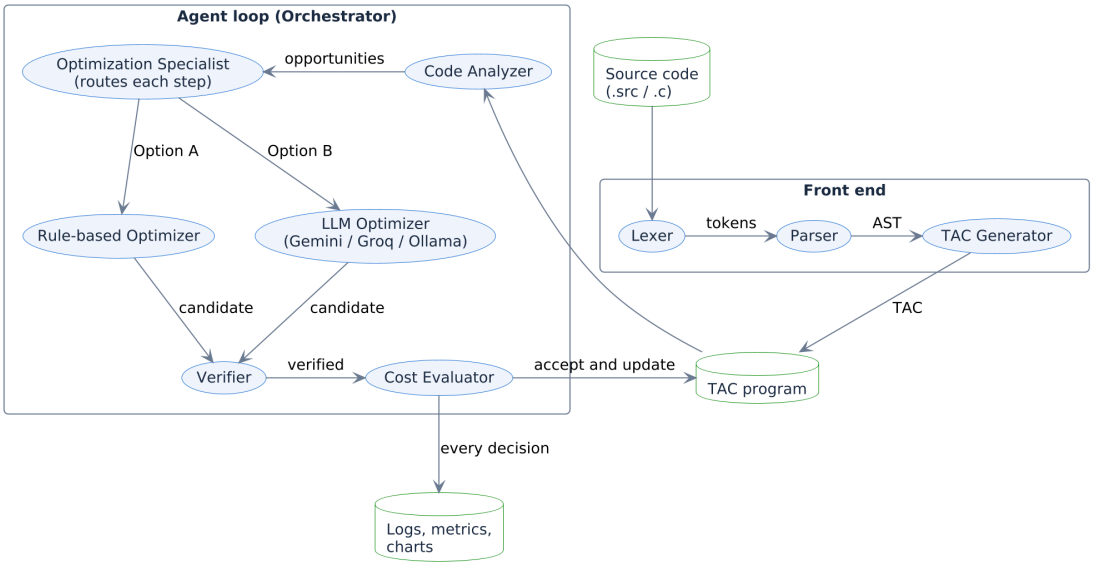


Figure 4.2: Data flow diagram showing inputs, outputs, and intermediate data at each stage

4.4.2 Class Diagram

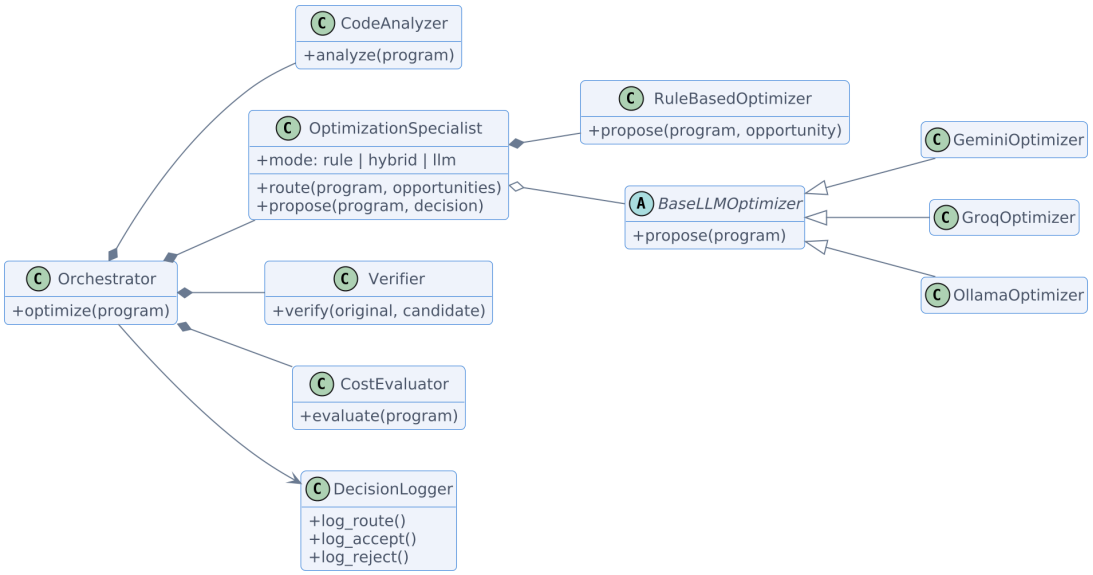


Figure 4.3: Class diagram of the agent architecture. The Optimization Specialist owns the rule-based optimizer and holds one of three interchangeable LLM backends

CHAPTER 5

IMPLEMENTATION

This chapter walks through the major modules, shows representative code fragments, and describes the test suite. All source code is in Python 3.10+.

5.1 MODULE DESCRIPTION

5.1.1 Module 1: Front End (`src/frontend/`)

The front end has four files: `lexer.py` (tokenization), `ast_nodes.py` (AST dataclasses), `parser.py` (recursive-descent parser), and `tac_generator.py` (AST-to-TAC lowering). A convenience function `compile_source(text)` chains the stages. A second front end, `c_frontend.py`, parses a subset of real C into the same AST, so an ordinary `.c` file can go through the same agents in the live demo.

Listing 5.1: From `tac_generator.py`: lowering an if statement that has an else part to TAC (trimmed)

```
def _gen_if(self, stmt: IfStmt) -> None:
    cond = self._gen_expr(stmt.condition)
    l_else = self._new_label()
    l_end = self._new_label()
    self._emit(TACInstruction(TACOp.IF_FALSE_GOTO, arg1=cond,
       label=l_else))
    self._gen_block(stmt.then_body)
    self._emit(TACInstruction(TACOp.GOTO, label=l_end))
    self._emit(TACInstruction(TACOp.LABEL, label=l_else))
    self._gen_block(stmt.else_body)
    self._emit(TACInstruction(TACOp.LABEL, label=l_end))
```

5.1.2 Module 2: TAC Representation and Interpreter (`src/tac/`)

The TAC subsystem defines `TACInstruction` (a dataclass with fields `op`, `result`, `arg1`, `operator`, `arg2`), `TACProgram` (a container with serialization, deep copy, and input-variable detection), and `TACInterpreter` (an execution engine that returns printed output as a list of integers). The interpreter enforces C-style truncating

division and raises typed exceptions (`DivisionByZeroError`, `ExecutionTimeout`, `UndefinedVariableError`) that the verifier distinguishes when comparing runs.

`dataflow.py` holds the CFG builder and the analyses: liveness, available copies, definite assignment, and reachability. All follow the textbook iterative fixed-point pattern.

5.1.3 Module 3: Agent Components (`src/agents/`)

The five components are separate classes. `orchestrator.py` runs the loop; `code_analyzer.py` finds opportunities; `optimization_specialist.py` routes each step and owns `optimizer_rule.py` and `optimizer_llm.py`; `verifier.py` and `cost_evaluator.py` are the two gates. Listing 5.2 is the core logic of the Orchestrator. It enforces our safety rules: a change is only kept if it passes verification *and* reduces the static cost. If the proposal for a step is turned down, the Specialist may hand the same step to the other proposer.

Listing 5.2: The Orchestrator loop (trimmed, from `orchestrator.py`)

```
for iteration in range(1, self.max_iterations + 1):
    # 1. ANALYZE (leaving out opportunities already turned down)
    opportunities = [o for o in self.analyzer.analyze(current)
                     if self._opportunity_key(o) not in attempted]

    # 2-5. PROPOSE -> VERIFY -> EVALUATE -> ACCEPT/REJECT
    tried, outcome = [], None
    while outcome not in ("accepted", "unverifiable"):
        decision = specialist.route(current, opportunities,
                                    llm_calls, tried)
        if decision.route is None:
            break                    # nobody can propose anything
                more
        tried.append(decision.route)
        proposal = specialist.propose(current, decision,
            opportunities)
        outcome = self._judge(iteration, current, proposal)
        if outcome == "accepted":
            current = proposal.candidate
            attempted.clear()        # new opportunities may
                appear

```

```
    if outcome != "accepted" and not opportunities:
        break                          # converged: nothing left to
            try
```

The verifier (`verifier.py`, 386 lines) runs differential testing first. If the program has no control flow, it also runs the symbolic trace and requires both to agree. The cost evaluator (`cost_evaluator.py`, 64 lines) counts instructions, arithmetic operations, temporary variables, memory operations (copies), and branch operations, multiplies each by a fixed weight (1.0, 2.0, 0.5, 1.5 and 1.0), and sums them into a single scalar.

5.1.4 Module 4: Optimization Specialist and Routing

The routing decision is a short method of the Optimization Specialist (Listing 5.3). The mode sets the order in which the two proposers are considered, and a proposer is skipped when it has nothing to offer.

Listing 5.3: The routing decision (trimmed, from `optimization_specialist.py`)

```
def route(self, program, opportunities, llm_calls=0, tried=()):
    order = ([Route.RULE, Route.LLM] if self.mode != "llm"
             else [Route.LLM, Route.RULE])
    if not self.llm_attached:
        order = [Route.RULE]
    passed_over, llm_note = [], ""
    for route in order:
        if route in tried:
            name = "rule-based" if route is Route.RULE else "LLM"
            passed_over.append(f"the {name} proposal was turned
                down")
        elif route is Route.RULE and not opportunities:
            passed_over.append("the rules have nothing left to try
                ")
        elif route is Route.LLM and (
                 llm_note := self._llm_blocker(program, llm_calls))
                    :
            passed_over.append(f"LLM not used, {llm_note}")
        else:
            reason = self._reason(route, opportunities,
                passed_over)
            return RoutingDecision(route, reason, llm_note)
```

```
19      return RoutingDecision(None, "; ".join(passed_over), llm_note)
```

5.1.5 Module 5: LLM Optimizer (`src/agents/optimizer_llm.py`)

The LLM optimizer defines a base class `BaseLLMOptimizer` with a shared prompt template and response parser, and three concrete subclasses: `GeminiOptimizer` calls the Google Gen AI SDK, `GroqOptimizer` calls Groq's HTTP API and moves to the next model when a daily quota is spent, and `OllamaOptimizer` calls a model running on the same machine. The prompt asks the model to propose "exactly one semantics-preserving optimization" and return the result inside `<tac>` tags. The parser strips markdown fences and line-number prefixes before feeding the text to `TACProgram.from_string()`.

The first prototype also had a provider for a paid API. It was removed, because a paid key made large-scale runs impractical; the three providers that remain are all free to use.

5.1.6 Module 6: Visualization and Demo

`charts.py` produces seven chart types: (1) Sankey diagram of proposal flow, (2) grouped bar chart of outcomes per optimization type, (3) line chart of cost trajectory for five representative programs, (4) donut chart of accepted optimization types, (5) scatter plot of agent steps vs. cost improvement with trend lines, (6) heatmap of category vs. optimization type, and (7) a comparison bar chart of rule-only vs. rule+LLM reduction per category. `report.py` aggregates per-program results into the metrics summary (JSON and CSV) and formats the human-readable metrics table. `demo.py` is a terminal front end that takes a C program and walks through the whole pipeline on screen, stage by stage.

5.2 TESTING

We wrote 336 automated tests across fifteen test files, covering every component (Table 5.1). The LLM backends are tested against stand-in local servers, so the tests need no API key, no Ollama installation and no network.

Table 5.1: Test suite summary

Test file	**Component**	**Tests**
`test_lexer.py`	Lexer	14
`test_parser.py`	Parser	22
`test_tac_generator.py`	TAC generator	14
`test_c_frontend.py`	C front end	43
`test_interpreter.py`	TAC interpreter	33
`test_analyzer.py`	Code analyzer	21
`test_optimizer.py`	Rule and LLM optimizers	27
`test_optimization_specialist.py`	Routing	20
`test_groq.py`	Groq backend	26
`test_ollama.py`	Ollama backend	24
`test_verifier.py`	Verifier	40
`test_cost_evaluator.py`	Cost evaluator	10
`test_orchestrator.py`	Orchestrator (integration)	16
`test_demo.py`	Demo front end	13
`test_bugfixes.py`	Regression tests	13

The verifier tests are worth calling out specifically. We inject incorrect transformations (wrong constant folds, operand swaps on non-commutative operators, deletion of live stores, off-by-one loop conditions, removed loop increments, deleted trapping divisions, reordered output, and dropped output) and confirm that every one is rejected. This is how we verify the verifier itself. We then tested the system at a larger scale:

- **Fuzzing the rules.** 5,100 random TAC programs were generated, and all 20,968 rule-based proposals for them were compared with the original on inputs the verifier does not use. None changed the program's behaviour.
- **Mutation testing the verifier.** Random branching programs were changed in one place (an operator, a constant or an operand), the way a wrong LLM proposal might be. Of the first 400 mutants, the verifier accepted 194, and 13 of those behaved differently from the original. After the small-scope and pairwise input sets were added, the verifier accepted 351 of 800 mutants and none of them behaves differently.

- **Re-checking the results.** For every run, each final program was executed against its original on every combination of small input values plus random ones (about 64,000 executions per run). No mismatch was found.
- **Replaying the Gemini run.** The 383 recorded Gemini replies were replayed through the strengthened verifier. Every proposal received the same decision as before.

The same testing found one bug in the rules: strength reduction rewrote $x \times 2^k$ to a shift even for $k > 64$, which the interpreter rejects. The proposal was caught by the verifier every time, and the analyzer no longer makes it.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 DETAILED RESULTS

We ran the agent four times on the full 500-program dataset: once with the rule-based optimizer only, and three times in `hybrid` mode, in which an LLM is asked when the rules hit a wall. Table 6.1 summarizes the headline numbers.

Table 6.1: Aggregate metrics across 500 programs

Metric	**Rule-only**	**Rule + Gemini**	**Rule + Groq**	**Rule + Ollama**
Total proposals	10,628	10,978	11,085	10,805
Accepted	10,517	10,828	10,925	10,579
Verification pass rate	100.00%	99.76%	99.95%	98.95%
Acceptance rate (of verified)	98.96%	98.87%	98.61%	98.94%
Avg. cost reduction	82.49%	91.50%	90.89%	83.28%
Steps to convergence (mean)	22.2	22.7	22.9	22.3
LLM validity rate	—	92.35%	98.11%	42.93%
False positives	**0**	**0**	**0**	**0**

The rule-based agent already meets the 30% cost-reduction target for every single program. Adding a cloud LLM lifts the average by roughly nine percentage points, with the biggest gains in the categories that rules handle least well (Table 6.2).

Table 6.2: Per-category cost reduction

Category	**Programs**	**Rule-only**	**+ Gemini**	**+ Groq**	**+ Ollama**
Arithmetic	60	96.03%	96.03%	96.03%	96.03%
Nested	60	98.10%	98.22%	98.31%	98.19%
Repeated expr	60	80.71%	81.65%	80.94%	80.71%
Dead code	60	97.44%	97.44%	97.44%	97.44%
Loops	80	65.05%	93.35%	93.43%	66.09%
Conditionals	80	73.91%	84.07%	83.48%	75.21%
Mixed	100	77.91%	91.56%	89.29%	79.96%
All	**500**	**82.49%**	**91.50%**	**90.89%**	**83.28%**

Arithmetic and dead-code programs are nearly fully reduced by rules alone, so there is not much room left for an LLM to help. Loops benefit the most (+28 percentage points with Gemini and with Groq) because the rule-based optimizer cannot evaluate a constant loop to a single value; it can fold constants inside the loop body, but the loop structure itself survives. An LLM, on the other hand, can reason that "this loop adds 3 eleven times, so the result is 33" and rewrite the entire loop as a single assignment.

6.1.1 Comparing the Three LLMs

Table 6.3 compares what each model contributed. No run made any program worse than the rule-only baseline.

Table 6.3: The three LLMs on the same 500 programs

	Gemini	**Groq**	**Ollama**
Runs on	Google cloud	Groq cloud	Laptop (CPU)
Programs improved over rules	196	181	32
Calls to the model	383	359	328
Usable proposals	340	318	198
accepted	262	248	60
correct, but no cheaper	52	64	25
wrong, blocked by the verifier	26	6	113
Validity rate	92.35%	98.11%	42.93%
Time for the full run	43 min	42 min	2 h 14 min

Gemini and Groq reach almost the same result. Groq's models were the most reliable: only 6 of 318 proposals were wrong. During the Groq run the first model (`gpt-oss-120b`) used up its free daily quota after 286 calls, and the system moved on to `gpt-oss-20b` for the last 148 programs without stopping. The local model was wrong more often than it was right, but because every proposal is verified this cost time and not correctness, and it still improved 32 programs with no internet connection at all.

6.2 OUTPUT SCREENSHOTS

6.2.1 Sample Run

Listing 6.1 shows the agent working on a small C program whose loop adds the same constant ten times. The rules clean up the temporaries and fold the constant; when they have nothing left to try, the step is routed to the LLM, which replaces the loop. The whole program becomes one instruction.

Listing 6.1: A real run of the demo on a C loop, with Groq as the LLM (shortened)

```
int sum = 0;
for (int i = 0; i < 10; i++) {
    int step = 2 + 3;
    sum += step * 1;
}
printf("%d\n", sum);

22 TAC instructions, weighted cost 56.0

[ 1] COPY_PROPAGATION  t0 = 0 -> uses of t0 -> 0
       verified  cost 56.0 -> 53.0  ACCEPTED
[ 6] CONSTANT_FOLD     t6 = 2 + 3 -> t6 = 5
       verified  cost 41.0 -> 40.5  ACCEPTED
...
[14] COPY_PROPAGATION  t11 = i + 1; i = t11 -> i = i + 1
       verified  cost 22.5 -> 19.5  ACCEPTED
     route to LLM optimizer: the rules have nothing left to try
[15] LLM (Groq)  the loop adds a constant 5 exactly ten times,
     so replace it with sum = 50 and eliminate the dead loop
       verified  cost 19.5 -> 3.5   ACCEPTED
```

```
[16] COPY_PROPAGATION  sum = 50 -> uses of sum -> 50
       verified  cost 3.5 -> 1.0    ACCEPTED

Optimized TAC:  print 50        cost 56.0 -> 1.0  (98.2% reduction)
original prints [50], optimized prints [50]: identical output
```

Figures 6.1 to 6.6 are drawn from the Gemini run. Figures 6.7 to 6.9 compare each LLM run with the rule-only baseline.

6.2.2 Sankey Diagram: Proposal Flow

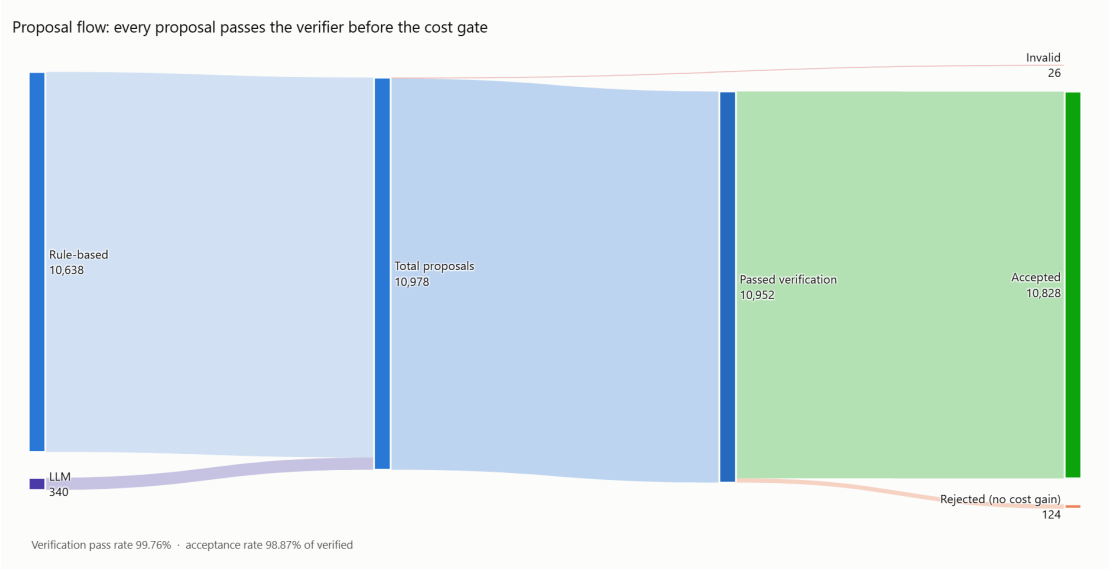


Figure 6.1: Sankey diagram showing the flow of 10,978 proposals from submission through verification to the accept/reject decision. The 26 invalid proposals are visible as a narrow band

6.2.3 Grouped Bar Chart: Outcomes by Optimization Type

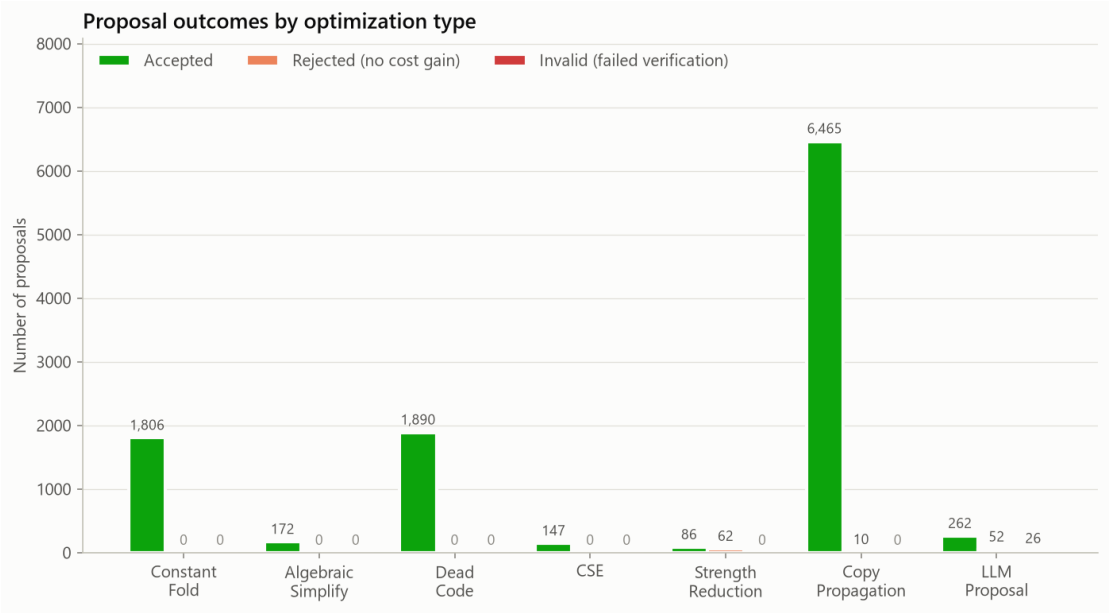


Figure 6.2: Accept, reject, and invalid counts for each optimization type. Copy propagation accounts for the most accepted changes

6.2.4 Line Chart: Cost Trajectory over Iterations

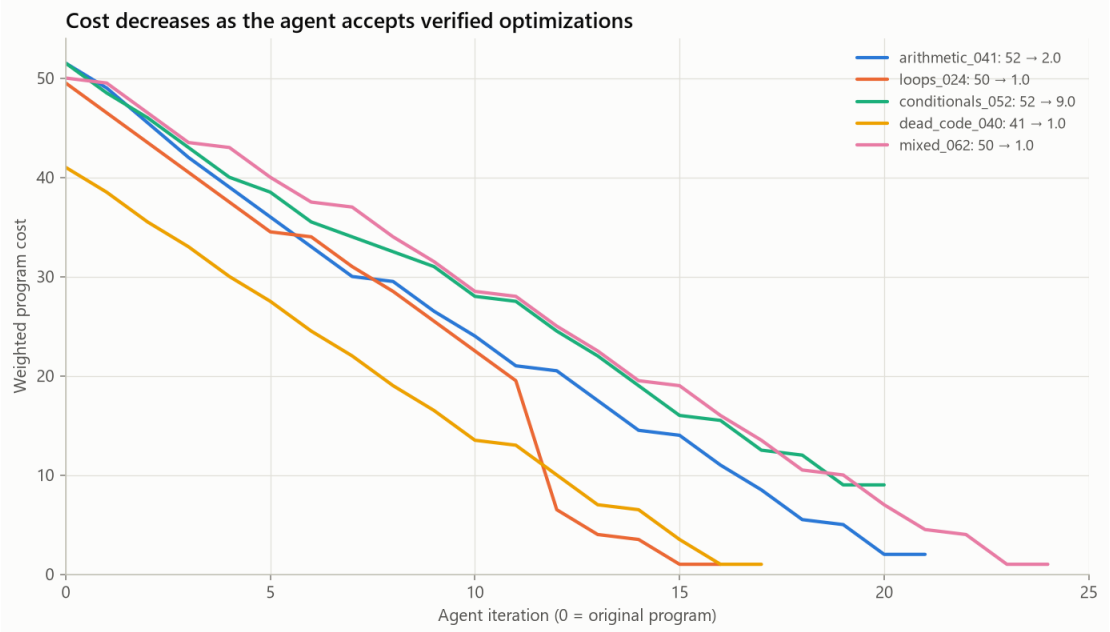


Figure 6.3: Weighted cost at each iteration for five representative programs. Cost decreases rapidly during early iterations and levels off as optimization converges

6.2.5 Pie Chart: Accepted Optimization Types

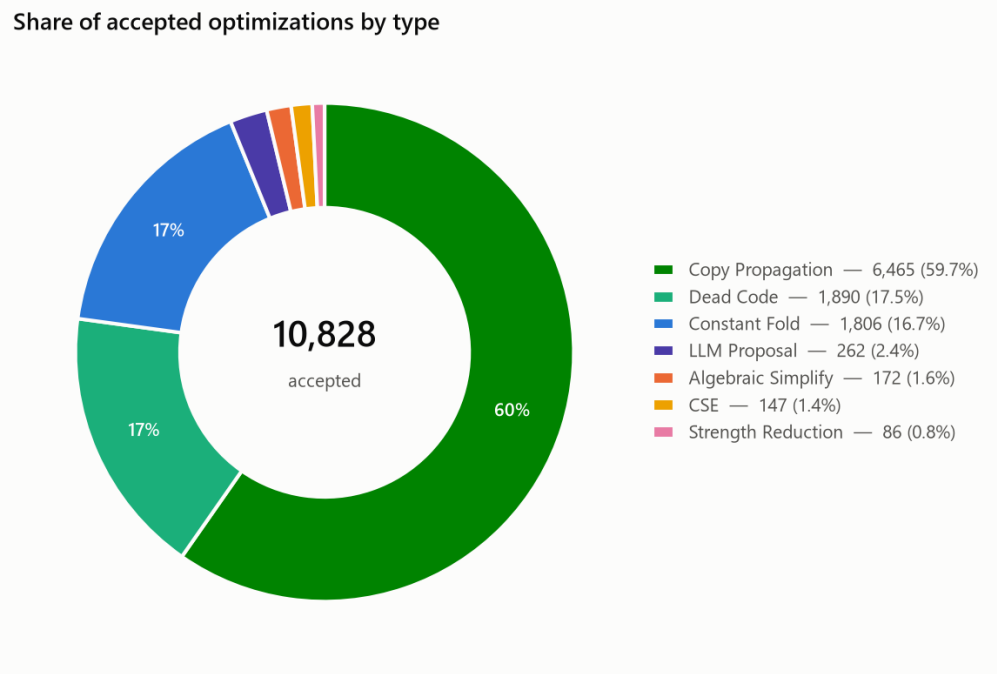


Figure 6.4: Breakdown of accepted optimizations by optimization type. Copy propagation accounts for the largest share of accepted changes

6.2.6 Scatter Plot: Steps vs. Cost Improvement

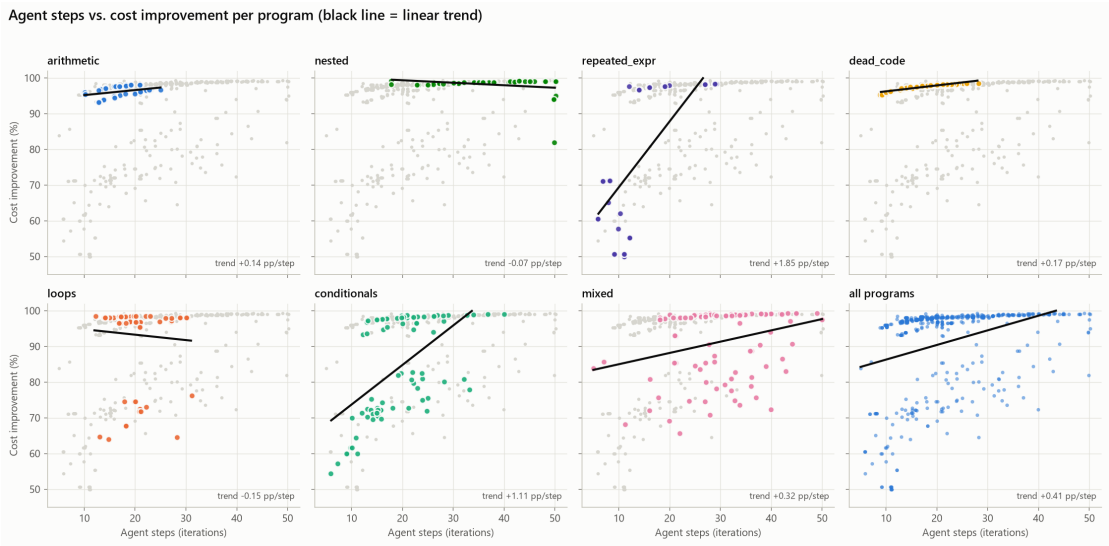


Figure 6.5: Agent steps versus percentage cost improvement across the program categories

6.2.7 Heatmap: Category vs. Optimization Type

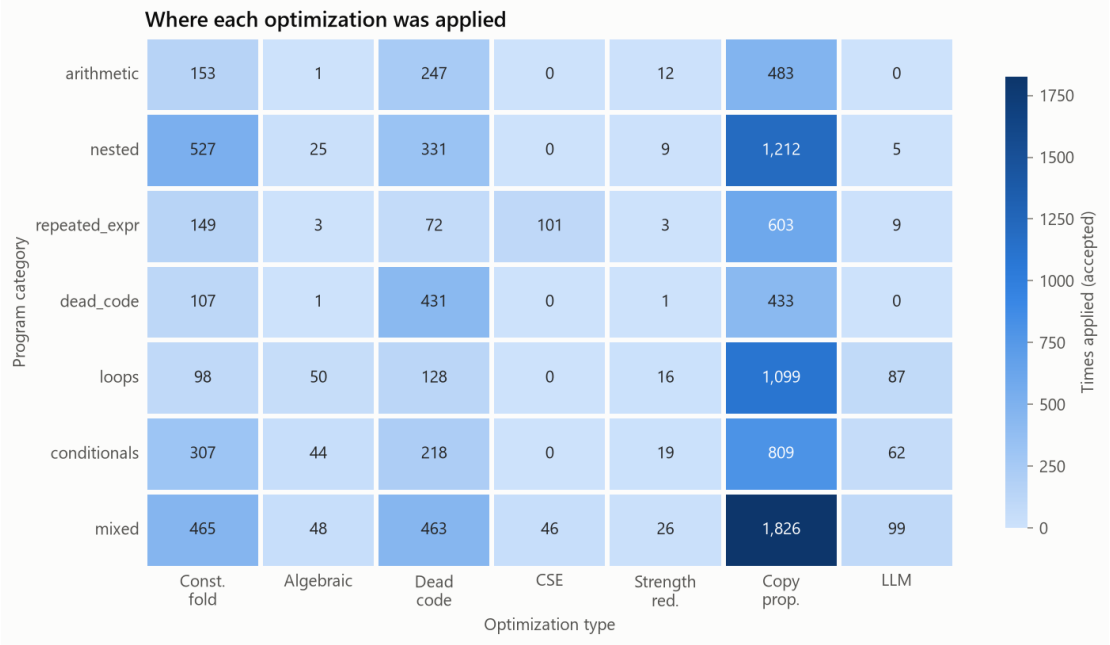


Figure 6.6: Number of accepted optimizations for each program category and optimization type

6.2.8 Comparison Charts: Rule-Only vs. Rule + LLM

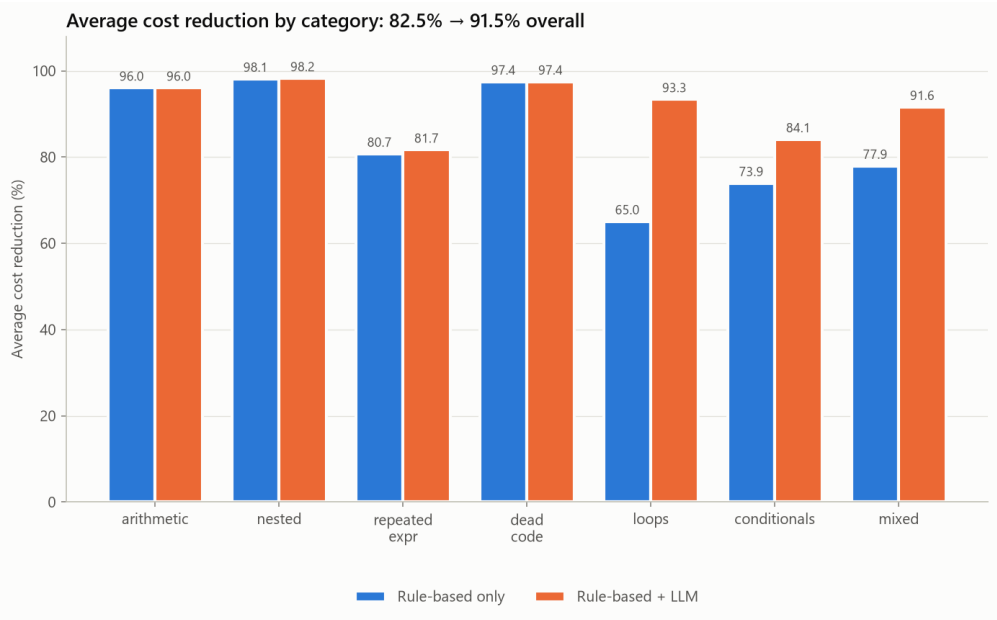


Figure 6.7: Average cost reduction per category comparing rule-only optimization with rule + Gemini. Loops rise from 65% to 93%

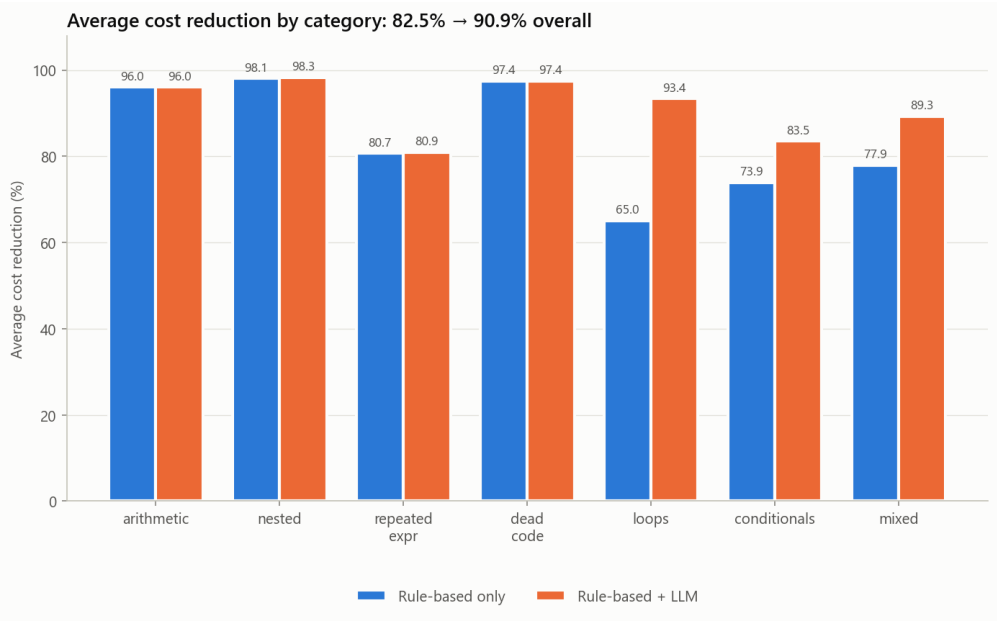


Figure 6.8: Rule-only against rule + Groq. The pattern is the same as with Gemini

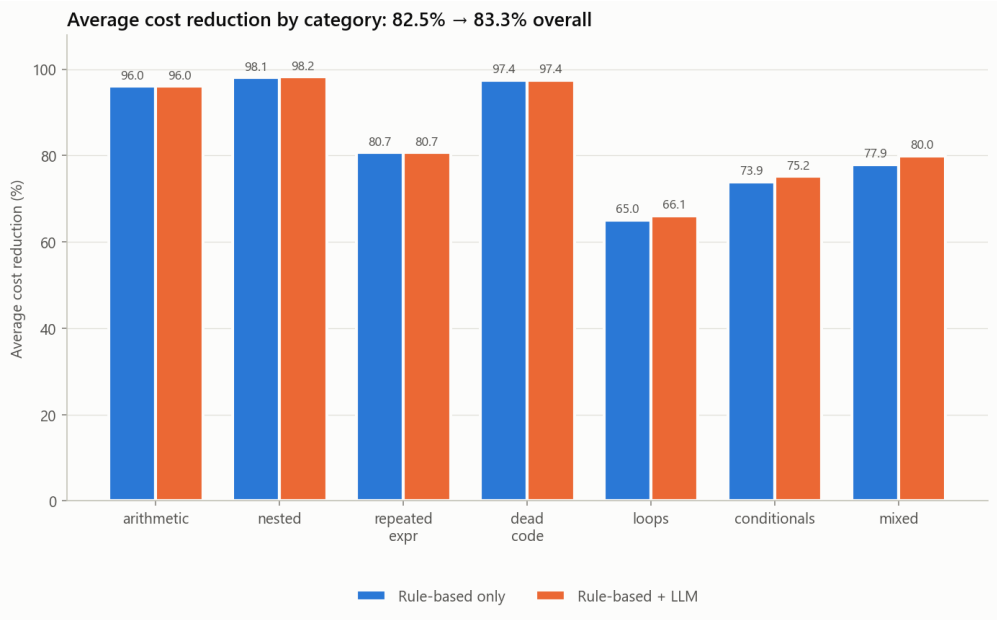


Figure 6.9: Rule-only against rule + the local Ollama model. The gains are small but come with no internet connection

6.2.9 Analysis Charts for the Groq Run

Figures 6.10 to 6.15 repeat the six charts for the run with Groq. The picture is close to the Gemini run: the same optimization types dominate, and only 6 proposals fail verification.

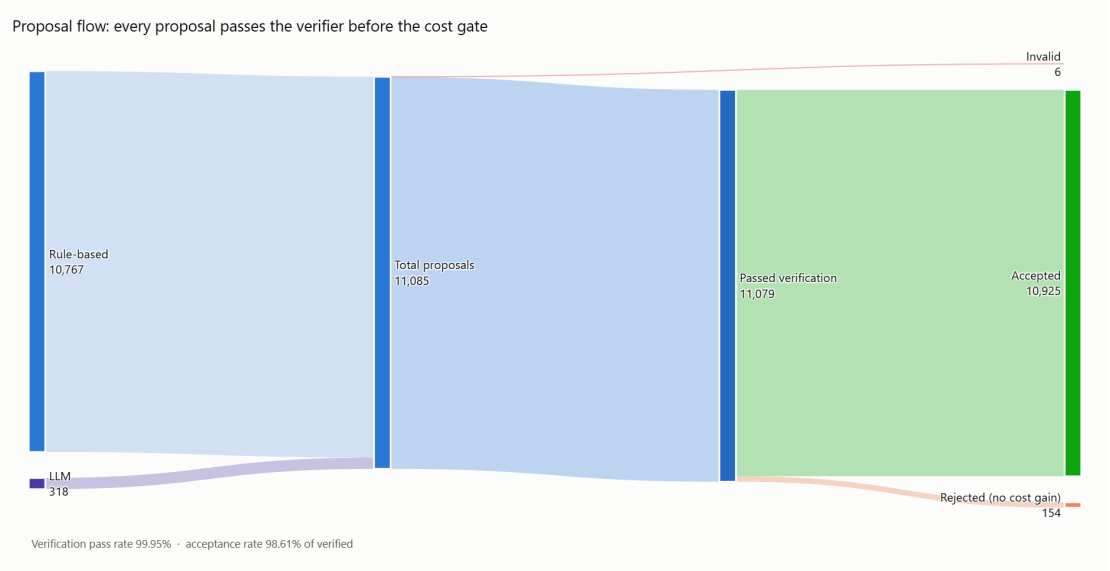


Figure 6.10: Groq run: flow of the 11,085 proposals. Only 6 LLM proposals were invalid, so the band of blocked proposals is barely visible

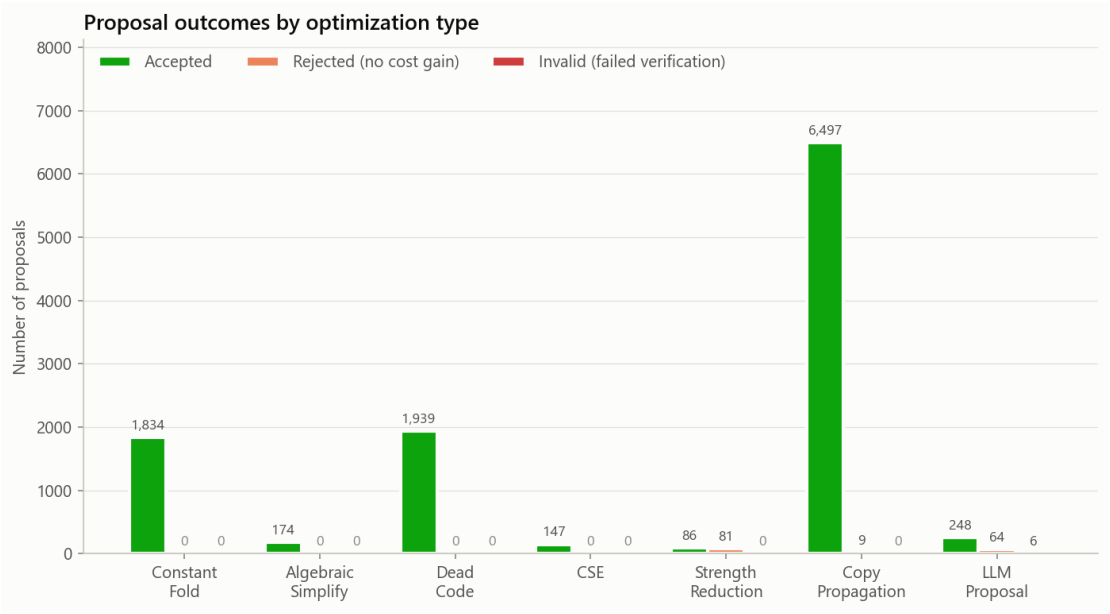


Figure 6.11: Groq run: accept, reject, and invalid counts for each optimization type

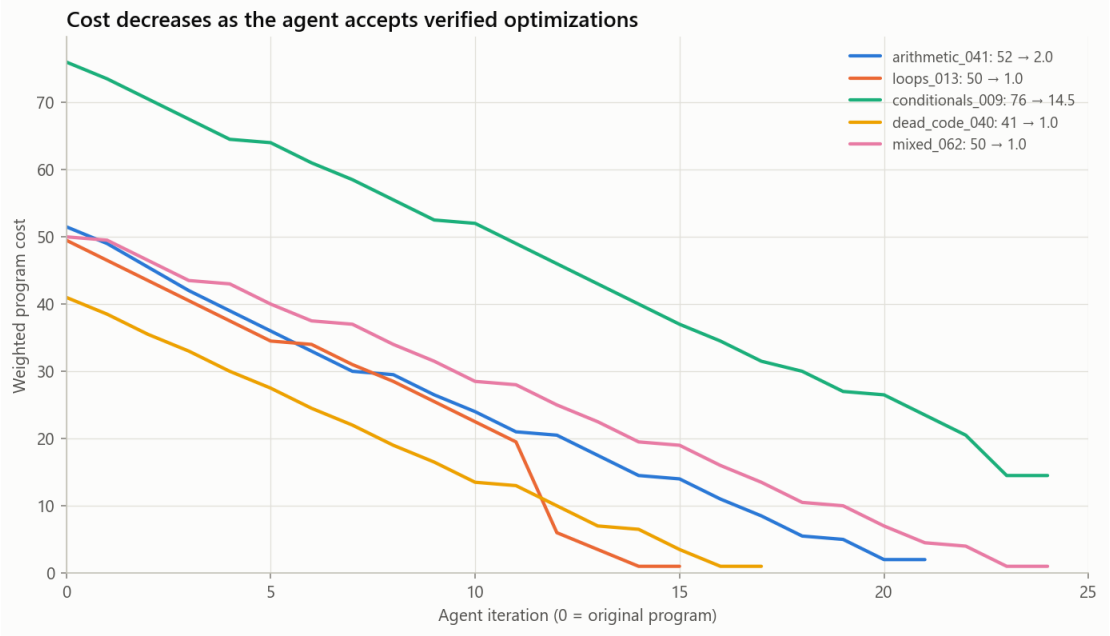


Figure 6.12: Groq run: weighted cost at each iteration for five representative programs

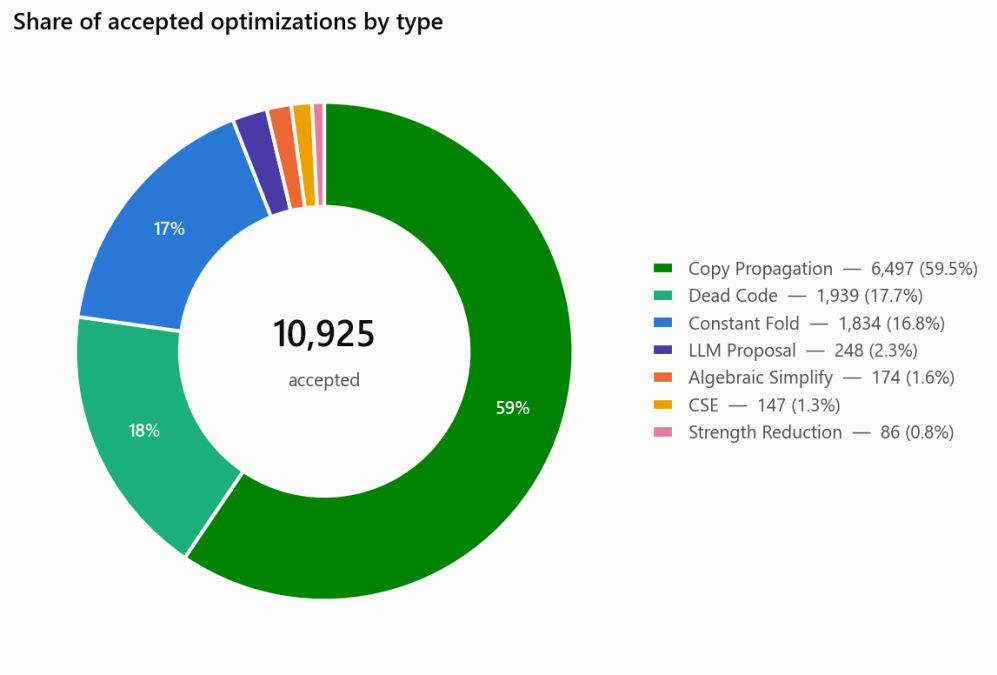


Figure 6.13: Groq run: breakdown of accepted optimizations by type

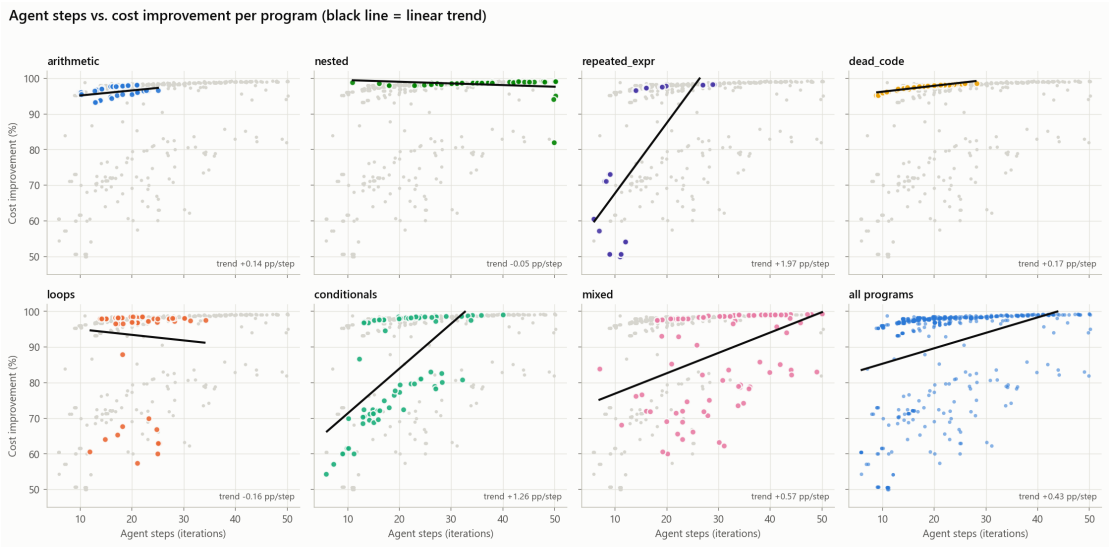


Figure 6.14: Groq run: agent steps versus percentage cost improvement

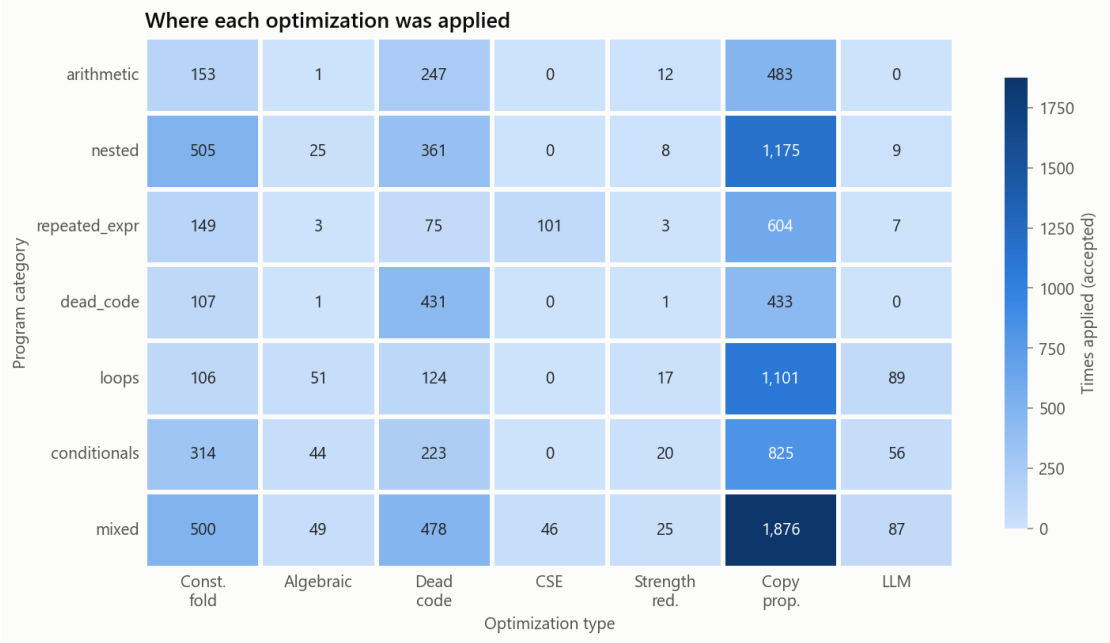


Figure 6.15: Groq run: accepted optimizations for each program category and optimization type

6.2.10 Analysis Charts for the Ollama Run

Figures 6.16 to 6.21 show the same charts for the offline run with the local model. Here the band of invalid proposals is clearly visible: 113 of the local model's proposals were wrong, and every one was blocked by the verifier.

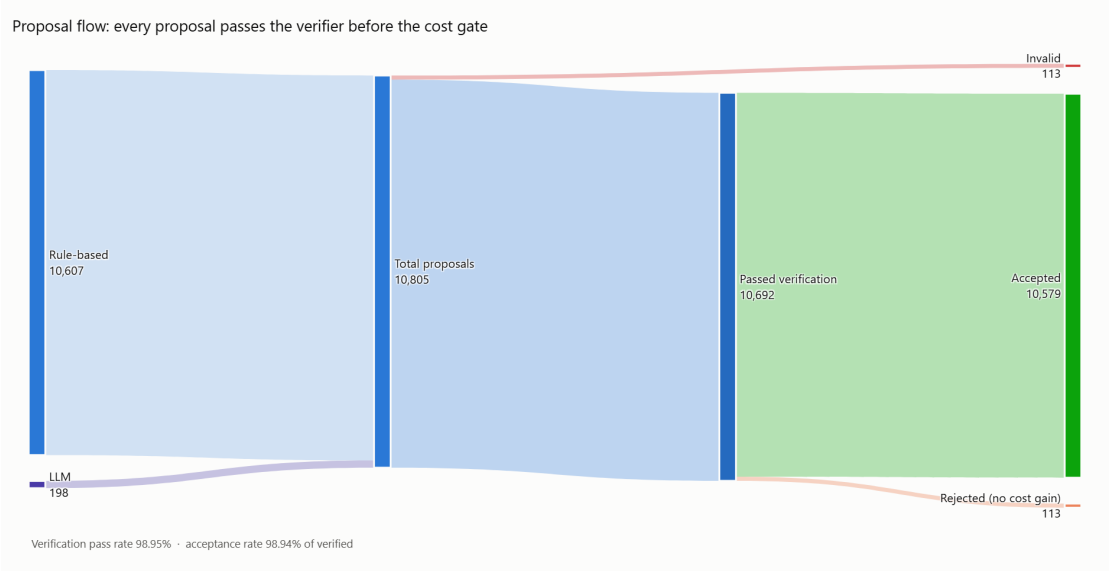


Figure 6.16: Ollama run: flow of the 10,805 proposals. The 113 invalid LLM proposals form a visible band that ends at the verifier

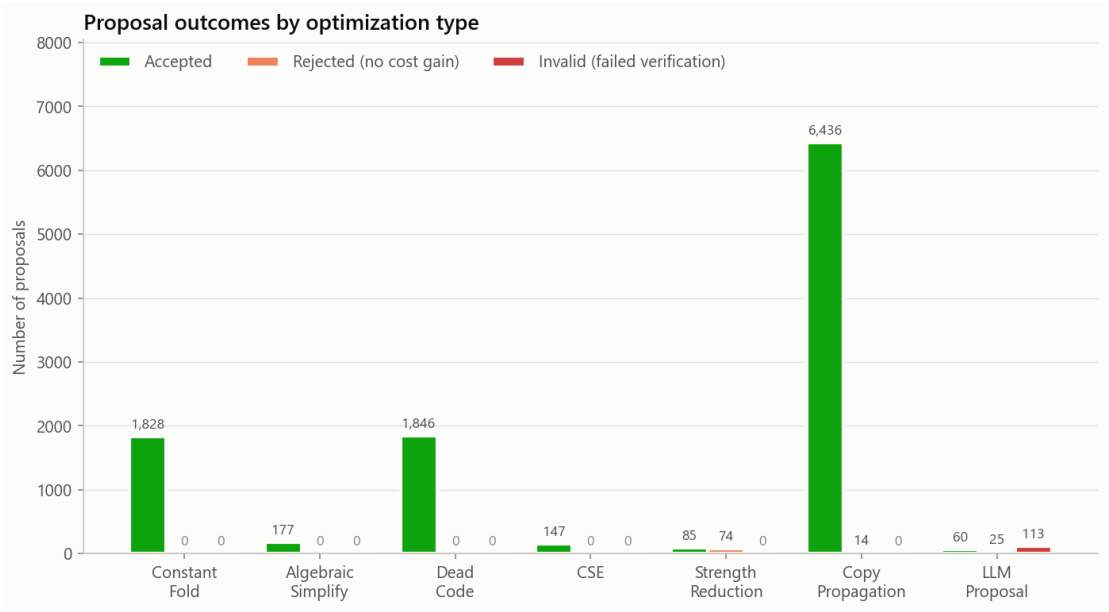


Figure 6.17: Ollama run: accept, reject, and invalid counts for each optimization type. The invalid bar belongs to the LLM proposals

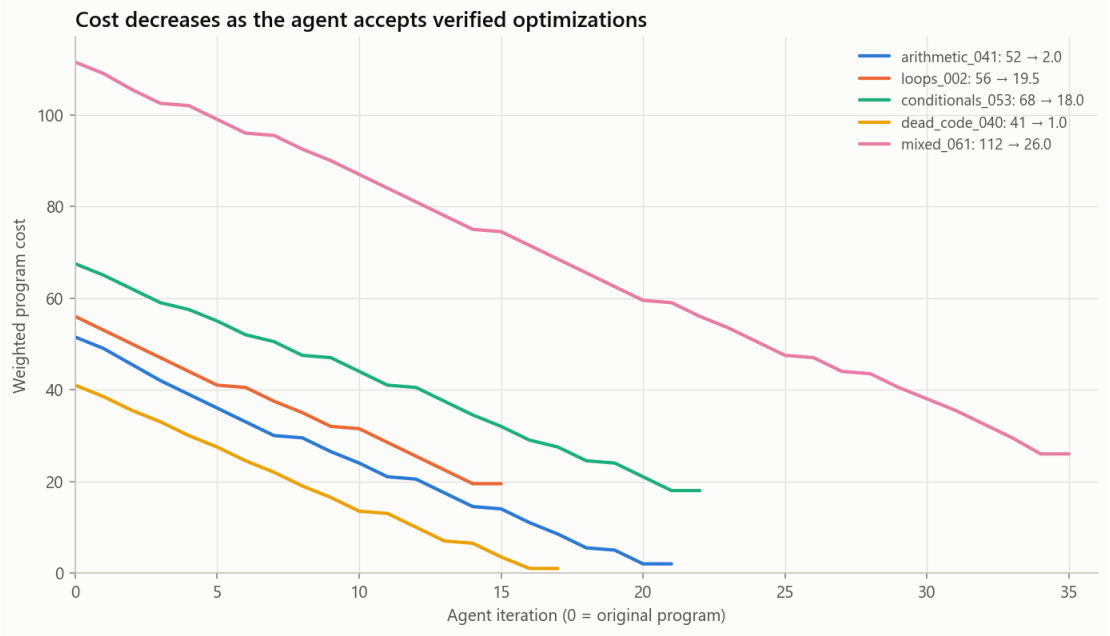


Figure 6.18: Ollama run: weighted cost at each iteration for five representative programs

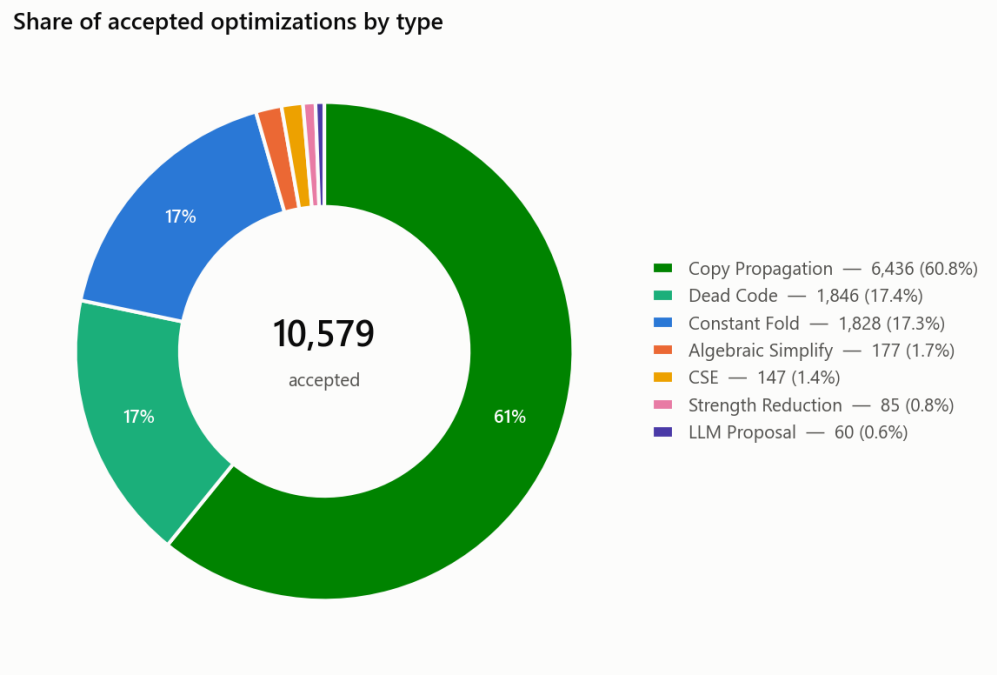


Figure 6.19: Ollama run: breakdown of accepted optimizations by type

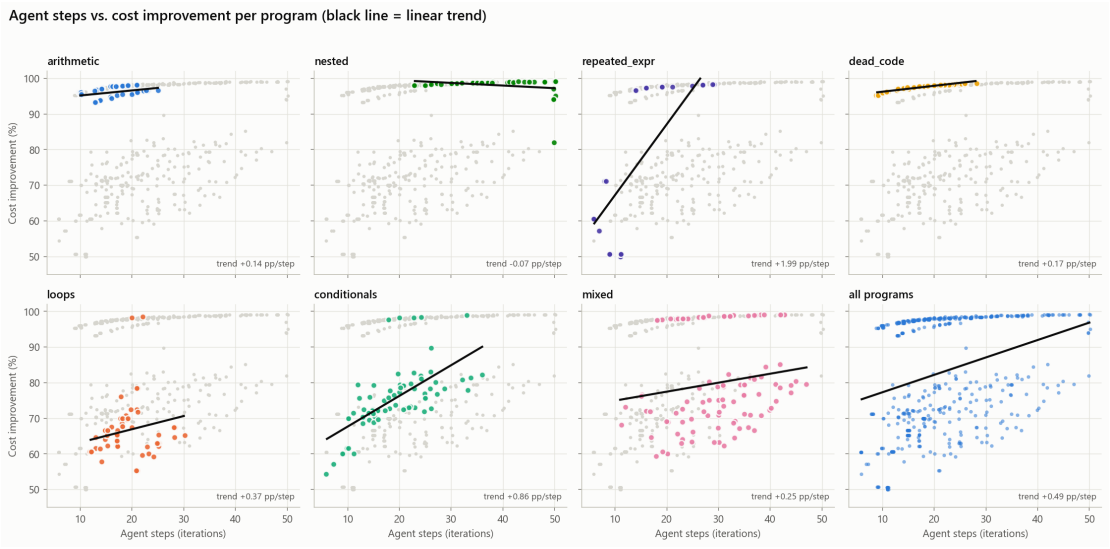


Figure 6.20: Ollama run: agent steps versus percentage cost improvement

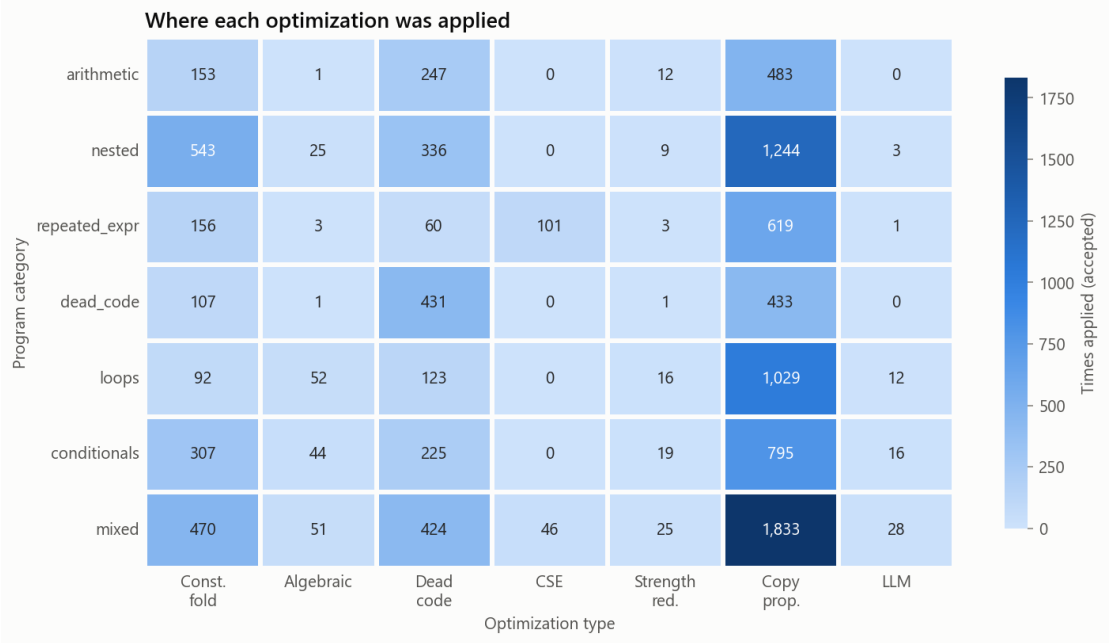


Figure 6.21: Ollama run: accepted optimizations for each program category and optimization type

6.3 PERFORMANCE ANALYSIS AND EVALUATION

A few observations stand out from the numbers:

The verifier works. All 10,628 rule-based proposals passed verification; not one was incorrect. This is expected, since the rules are correct by construction, but it confirms that the verifier does not produce false rejections. On the LLM side, 26 of Gemini's 340 usable proposals (7.65%), 6 of Groq's 318 (1.9%) and 113 of the local model's

198 (57%) were semantically wrong, and the verifier caught every single one. The false-positive count across all four runs is zero.

Cost-neutral strength reductions get rejected, as intended. Under our weight scheme, replacing `x * 2` with `x + x` swaps one arithmetic operation for another at the same cost. The evaluator rightly rejects these. Replacing `x * 4` with `x << 2`, on the other hand, replaces an arithmetic operation (weight 2.0) with a non-arithmetic shift, so it is accepted. This accounts for 97 of the 111 rejections in the rule-only run.

Copy propagation is the workhorse. It accounts for about 60% of all accepted changes because each high-priority transformation (a fold, a CSE, a dead-code removal) often creates new copies that need to be propagated and cleaned up. A single constant fold might trigger three or four subsequent copy-propagation steps.

The LLM helps most where rules are structurally limited. Rules cannot "think about" what a loop computes in aggregate. They can fold constants inside the body, but the loop control flow survives. An LLM can recognize that a loop with a constant trip count and a constant body evaluates to a single value, and it proposes the collapsed form. The verifier then confirms it on its test inputs. This pattern explains the 28-point jump in the loops category.

A weak model is still safe. The local model's low validity rate did not make the results worse, only slower. This is the main practical consequence of verifying every proposal: the choice of model affects how much is gained, never whether the result is correct.

Convergence is fast. The median program converges in 20 iterations (about 0.5 seconds rule-only). The full 500-program rule-only run finishes in under 2 minutes. With a cloud LLM the bottleneck is the pause between API calls, which the free tiers require, bringing the total to about 43 minutes. Both cloud providers also limit requests and tokens per day; the system spaces its calls, waits out per-minute limits, and for Groq moves to another model when one quota is spent, so a full run completes without supervision. The local model has no quota but took 2 hours 14 minutes on the laptop CPU.

6.3.1 Limitations

- The cost model is static: it counts instructions, arithmetic operations, temporaries, copies and branches. It does not measure execution time, so removing an instruction inside a loop is valued the same as removing one outside it.
- For programs with branches *and* inputs, differential testing is thorough but is not a proof. Programs without inputs and straight-line programs are proven equivalent.
- The offline run used the CPU, because the local GPU driver was too old for the model runner. With a working GPU the same run should be several times faster.
- The dataset programs are small and generated. Larger, real-world programs remain future work.

REFERENCES

Journals and Preprints:

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P., Kaplan, J., … & Zaremba, W. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G., & Leather, H. (2024). Meta Large Language Model Compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524*.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., … & Synnaeve, G. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., … & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Conferences:

Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., … & Leather, H. (2022). CompilerGym: Robust, performant compiler optimization environments for AI research. *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 92–105.

Lattner, C., & Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis & transformation. *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 75–86.

Massalin, H. (1987). Superoptimizer: A look at the smallest program. *Proceedings of the 2nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 122–126.

Necula, G. C. (2000). Translation validation for an optimizing compiler. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 83–94.

Phothilimthana, P. M., Thakur, A., Bodik, R., & Dhurjati, D. (2016). Scaling up superoptimization. *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 297–310.

Pnueli, A., Siegel, M., & Singerman, E. (1998). Translation validation. *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 151–166.

Books:

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.

Muchnick, S. S. (1997). *Advanced Compiler Design and Implementation*. Morgan Kaufmann.

Stallman, R. M., & the GCC Developer Community. (2023). *Using the GNU Compiler Collection (GCC 13)*. Free Software Foundation.

Web links:

Groq. (2026). *GroqCloud API documentation*. https://console.groq.com/docs

Ollama. (2026). *Ollama: Run large language models locally*. https://ollama.com